

Universidad Distrital Francisco José de Caldas

Facultad de Ingeniería – Computer Science I (2026-I)

Tower Siege

A Grid-Based Tower-Defense Engine Demonstrating
Linked Lists, Binary Search Trees, Greedy Placement,
and Backtracking Pathfinding

Daniel Santiago Pérez Madera

Codigo: 20231020203

`dsperezm@udistrital.edu.co`

June 2026

Abstract

Tower-defense games provide a natural laboratory for evaluating how classical algorithms behave under real-time constraints and adversarial inputs. This report presents *Tower Siege*, a two-layer system in which a C++ engine manages game state using hand-implemented data structures—a singly linked list for the active enemy wave and a Binary Search Tree for towers ordered by attack power—while a Python layer runs a *Greedy* tower-placement advisor and a *Backtracking* pathfinder, exchanging data exclusively through JSON files. The project achieves a fully playable five-level game with two tower types, two enemy types, real-time HUD feedback, and algorithm visualisation; however, a systematic interaction between the Greedy scoring function and the Backtracking DFS was identified in which the Greedy’s castle-proximity bias progressively narrows enemy corridors until the Backtracking’s exponential search space causes perceptible frame-rate degradation, an issue that is analysed in depth and whose remediation is proposed as future work.

Contents

1	Introduction	3
2	Literature Review	4
2.1	Greedy Algorithms	4
2.2	Backtracking and Pathfinding	4
2.3	Data Structures in Game Engines	4
3	Objectives	5
3.1	General Objective	5
3.2	Specific Objectives	5
4	Scope	6
5	Assumptions	7
6	Limitations	8
7	Methodology	9
7.1	System Architecture	9
7.2	Data Structures	10
7.2.1	Enemy Wave: Singly Linked List	10
7.2.2	Tower Registry: Binary Search Tree	10
7.3	Algorithms	10
7.3.1	Greedy Tower Placement	10
7.3.2	Backtracking Pathfinding	10
7.4	JSON Communication Contract	11
7.5	Level Design	11
8	Implementation & Results	12
8.1	Development Environment	12
8.2	C++ Engine	12
8.3	Python Layer	12
8.4	Screenshots	13
8.5	Validation Results	13
9	Discussion	16
9.1	Greedy–Backtracking Interaction Bug	16

9.1.1	Root Cause: Castle-Proximity Bias in Greedy	16
9.1.2	Pathfinder Exposure	16
9.1.3	Observable Symptom	17
9.1.4	Proposed Remediations	17
9.2	Data Structure Performance	17
10 Conclusion		19
References		20

1. Introduction

Tower-defense games represent a category of real-time strategy in which the player places static defensive units (towers) on a grid to prevent waves of enemies from reaching a goal cell (the castle). Beyond entertainment, this genre is a useful proxy for scheduling, resource allocation, and pathfinding problems that appear in operations research and algorithmic decision systems [1].

Prior solutions to in-game pathfinding range from simple breadth-first search to A^* with domain-specific heuristics [2]. Tower-placement advice has been addressed through influence maps, potential fields, and greedy heuristics that score candidate cells by their proximity to high-traffic corridors. The tension between placement strategies and pathfinding robustness—each tower added reshapes the search space for path computation—motivates a direct study of how these two components interact when integrated in the same game loop.

Tower Siege was designed as an academic vehicle to implement and stress-test four specific algorithmic and data-structural artefacts within a single coherent application:

1. A **singly linked list** (C++) maintaining the active enemy wave.
2. A **Binary Search Tree** (C++) indexing towers by attack power.
3. A **Greedy** heuristic (Python) recommending the best tile for the next tower.
4. A **Backtracking DFS** (Python) computing the shortest obstacle-free path from each spawn point to the castle.

The two computational layers communicate exclusively via JSON files on disk, providing a clean inter-process boundary that mirrors real-world service architectures [3]. The rest of this report is structured as follows: Chapter 2 surveys related literature; Chapter 3 states the project goals; Chapters 4–6 define scope, assumptions, and limitations; Chapter 7 describes design decisions and architecture; Chapter 8 presents the implemented system and test results; Chapter 9 analyses the identified Greedy–Backtracking interaction bug; and Chapter 10 summarises findings and proposes future work.

2. Literature Review

2.1 Greedy Algorithms

A greedy algorithm builds a solution incrementally by always making the choice that looks best at the current step without revisiting past decisions [1]. The optimality guarantee holds when the problem exhibits the *greedy-choice property* and *optimal substructure*; in general combinatorial settings these conditions may not hold, and greedy produces an approximation rather than an exact optimum [1]. In game-algorithm design, greedy heuristics are widely used for tower placement because they run in $O(n \cdot m)$ time over an $n \times m$ grid and provide instantaneous visual feedback, a requirement for real-time systems. A common scoring approach combines enemy proximity, path frequency (influence maps), and goal proximity; the choice of weights critically determines whether the advisor steers the player toward effective or counterproductive placements [1].

2.2 Backtracking and Pathfinding

Backtracking is a systematic DFS strategy that extends partial candidates and prunes branches that cannot lead to a valid solution [1]. For grid pathfinding without heuristic guidance it is equivalent to exhaustive DFS, with worst-case complexity $O(4^{n \cdot m})$ on an $n \times m$ grid [1]. Practical pathfinding in games typically uses A^* , which combines a forward cost with an admissible heuristic to achieve $O(b^d)$ complexity where b is the branching factor and d the optimal path depth [2]. Backtracking-based pathfinding is nevertheless instructive in academic settings because it exposes how the search space grows with grid occupancy—a property exploited in the bug analysis of Chapter 9.

2.3 Data Structures in Game Engines

Linked lists are the canonical container for dynamic entity sets where frequent insertions and deletions occur at arbitrary positions, as in an enemy wave queue [1]. Binary Search Trees provide $O(\log n)$ average-case search and sorted traversal, making them appropriate for ordering towers by attack power to display a real-time ranking [1]. Both structures are implemented here from scratch, following course requirements that prohibit the use of STL equivalents such as `std::list` or `std::map`.

3. Objectives

3.1 General Objective

To design and implement a two-language, file-coupled tower-defense game that demonstrates the application of hand-rolled data structures (linked list and BST) and algorithmic strategies (Greedy and Backtracking) within a real-time, interactive context.

3.2 Specific Objectives

1. Implement a singly linked list in C++ to manage the active enemy wave, supporting $O(1)$ append, $O(n)$ removal by ID, and sequential traversal for advance and attack resolution.
2. Implement an unbalanced BST in C++ to maintain towers ordered by attack power, supporting $O(h)$ insertion and $O(n)$ in-order traversal exported to the Python layer via JSON.
3. Write a Greedy function in Python that scores every free cell of the grid and returns the cell with the highest incoming-threat score as a placement recommendation.
4. Write a Backtracking DFS in Python that finds the shortest path from each spawn point to the castle cell, avoiding tower-occupied cells.
5. Integrate both layers through a JSON file bridge so that all data exchange occurs through `input.json` (Python $\rightarrow$ C++) and `state.json` (C++ $\rightarrow$ Python).
6. Build a Pygame UI that visualises algorithm decisions—highlighted greedy recommendation, drawn enemy paths, and BST-sorted tower list—and supports five game levels of increasing difficulty.

4. Scope

Tower Siege is scoped as a single-player, offline, grid-based tower-defense game running on a local machine. The project encompasses:

- A C++ engine binary that reads one JSON command per invocation and writes one JSON state snapshot (stateless per-call model).
- A Python game loop (Pygame) that is the sole entry point (`game/ui/main.py`).
- Five levels with grids of 8×8 (levels 1–2) and 10×10 (levels 3–5), two tower types, two enemy types, and up to two spawn points.
- Algorithm visualisation: the greedy-recommended cell is highlighted in teal; enemy paths drawn as blue outlines; tower attack lines rendered each step; the BST order shown in the HUD panel.

5. Assumptions

1. The C++ engine binary is compiled before the game is launched (`tower_engine.exe` present at `engine/tower_engine.exe`).
2. Python 3.9 or later and Pygame 2.x are installed in the execution environment.
3. The grid is always square (`rows = cols`); non-square grids are undefined behaviour.
4. Enemy HP values are always positive integers; the engine does not validate input ranges beyond assertions.
5. Each enemy follows exactly one pre-computed path identified by its `path_id` field; path recomputation happens only when the tower layout changes.
6. Manhattan distance is the appropriate proximity metric for both tower attack range and greedy scoring, consistent with 4-directional grid movement.
7. The castle cell is always at position $(\text{grid} - 1, \text{grid} - 1)$, and spawn points are always in row 0.

6. Limitations

1. **Backtracking complexity.** The pathfinder uses exhaustive DFS with a pruning optimisation (branch pruned when the partial path plus the Manhattan lower bound to the goal already exceeds the current best). Worst-case complexity remains $O(4^{n \cdot m})$, which becomes perceptible on heavily occupied grids (see Chapter 9).
2. **Unbalanced BST.** The BST does not implement rotations; degenerate insertion order (e.g., towers placed in strictly increasing attack-power order) reduces the tree to a linked list with $O(n)$ search.
3. **Greedy sub-optimality.** The Greedy scoring function overweights castle proximity, causing systematic recommendations of near-castle cells that conflict with effective tower-defense strategy (detailed in Chapter 9).
4. **Stateless engine.** The C++ engine reconstructs all data structures from the JSON payload on every invocation. For large grids or long waves this introduces unnecessary serialisation overhead.
5. **No adaptive opponent.** Enemy routing is reactive (follows shortest available path) rather than adversarial; enemies do not adapt to tower placements beyond the path recomputation.
6. **Single platform.** The engine binary path is hard-coded for Windows (`tower_engine.exe`); a Makefile with a platform-detection variable would be required for Linux/macOS builds.

7. Methodology

7.1 System Architecture

Tower Siege uses a strict three-layer architecture mandated by the course assignment. The layers are decoupled by a file-based inter-process communication (IPC) contract so that neither layer depends on the other's runtime.

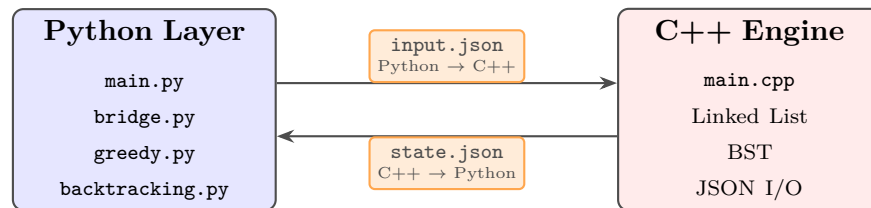


Figure 7.1: Three-layer architecture of Tower Siege. The Python layer writes `input.json` (player command) and reads `state.json` (updated game state); the C++ engine does the inverse.

Table 7.1 summarises each layer's language, files, and primary responsibility.

Table 7.1: Layer responsibilities in Tower Siege.

Layer	Language	Key files	Responsibility
Core Engine	C++	<code>main.cpp</code> , <code>linked_list.cpp</code> , <code>tree.cpp</code>	Maintains enemy wave (linked list) and tower set (BST). Reads <code>input.json</code> , writes <code>state.json</code> .
Algorithm	Python	<code>greedy.py</code> , <code>backtracking.py</code>	Greedy tile scoring and backtracking pathfinding. Pure functions; no I/O.
UI / Bridge	Python	<code>main.py</code> , <code>bridge.py</code>	Pygame rendering, user input, level management, JSON exchange.

7.2 Data Structures

7.2.1 Enemy Wave: Singly Linked List

The active enemy wave is stored as a singly linked list (`EnemyList` in `linked_list.h`). A dedicated `tail` pointer keeps append (`ll_push_back`) at $O(1)$. Removal of a dead enemy (`ll_remove_id`) requires an $O(n)$ scan because the list is unordered by ID. Each step, `ll_advance` walks the list once and increments each enemy's position along its pre-computed path by `speed` cells.

7.2.2 Tower Registry: Binary Search Tree

Towers are inserted into an unbalanced BST (`TowerTree` in `BSTree.h`) keyed on `attack_power`. In-order traversal (`tt_inorder`) yields towers sorted from weakest to strongest; this sorted list is serialised into `state.json` and displayed in the HUD panel to demonstrate the BST property visually. Insertion is $O(h)$ where h is tree height; with random attack-power values $h \approx \log n$ on average.

7.3 Algorithms

7.3.1 Greedy Tower Placement

`greedy.recommend_tile` iterates over every free cell of the grid and computes a scalar threat score:

$$S(r, c) = \sum_{\substack{e \in \text{enemies} \\ d_e \leq 7}} (8 - d_e) \frac{e.\text{hp}}{100} + \max(0, 8 - d_{\text{castle}}) \cdot 1.5$$

where $d_e = |r - e_r| + |c - e_c|$ is the Manhattan distance to enemy e and d_{castle} is the Manhattan distance to the castle. The cell with the highest score is returned as the recommended placement and highlighted in teal on the grid. Complexity is $O(n \cdot m \cdot |\text{enemies}|)$.

7.3.2 Backtracking Pathfinding

`backtracking.find_path` performs DFS from each spawn point to the castle cell, exploring the four cardinal neighbours and pruning branches whenever the partial path length plus the Manhattan lower bound to the goal already meets or exceeds the current best-found path length. The function returns the shortest path (minimum number of steps) or an empty list if no path exists. `game/ui/main.py` also uses this function in `placing_blocks_path` to prevent the player from sealing off all enemy routes before placing a new tower.

7.4 JSON Communication Contract

The engine reads `data/input.json` and writes `data/state.json` on each invocation (Table 7.2). The C++ engine implements its own JSON serialiser/parser using only the standard library, as required by course constraints—the `nlohmann/json` library [4] was considered but excluded because external C++ dependencies are forbidden by the assignment.

Table 7.2: Top-level fields in the JSON contract.

File	Field	Description
input.json	command	One of "init", "step", "place_tower", "remove_tower"
	grid	{rows, cols} – grid dimensions
	castle	{row, col} – castle position
	castle_hp	Current castle hit points
	towers	Array of tower objects
	enemies	Array of enemy objects
	paths	Array of paths (one per spawn); each path is an array of {row,col} waypoints
state.json	castle_hp	Updated castle HP after the step
	towers_by_power	Towers in ascending attack-power order (BST in-order)
	enemies	Updated enemy array
	events	Events emitted: tower_attack, enemy_killed, castle_damaged, wave_clear

7.5 Level Design

Five levels of escalating difficulty are defined in `levels.py` (Table 7.3). Levels 1–2 use an 8×8 grid; levels 3–5 use 10×10 . Level 5 introduces a second spawn point at (0,9), requiring the Backtracking to compute two independent paths simultaneously.

Table 7.3: Level configuration summary.

Level	Grid	Enemies	Spawn ms	Tower budget	Heavy?	Spawns
1	8×8	5	4000	4	No	1
2	8×8	6	3500	4	No	1
3	10×10	7	3500	5	Yes	1
4	10×10	8	3000	5	Yes	1
5	10×10	10	3000	6	Yes	2

8. Implementation & Results

8.1 Development Environment

The project was developed on Windows 11 using:

- **C++17** compiled with `g++ -std=c++17 -O2` (MinGW-w64).
- **Python 3.12** with **Pygame 2.x** [5].
- No third-party C++ libraries; JSON parsing implemented by hand in `engine/JsonIO.cpp`.

8.2 C++ Engine

The engine (`engine/main.cpp`) executes the following flow on each invocation:

1. Parse `input.json` with `read_input()`.
2. Rebuild the `EnemyList` and `TowerTree` from the deserialized vectors.
3. Apply the requested command. For a "step" command: advance all enemies one tick (`ll_advance`), resolve tower attacks (`apply_tower_attacks`), resolve castle damage (`resolve_castle_damage`). Other commands echo state without modifications.
4. Serialise the resulting `GameState` to `state.json` with `write_state()`.

Tower attacks are resolved from weakest to strongest (in-order BST traversal); each tower targets the *nearest* alive enemy within its Manhattan radius.

8.3 Python Layer

The Python layer consists of four modules:

- `bridge.py` – writes `input.json`, launches the engine via `subprocess.run`, reads `state.json`.
- `greedy.py` – pure function; returns recommended cell.
- `backtracking.py` – pure function; returns path as list of waypoints.
- `levels.py` – pure data; defines `LEVELS`, `TOWER_TYPES`, and `ENEMY_TYPES`.

The Pygame UI (`main.py`) runs at 30 FPS with a 1-second engine step interval and a per-level configurable enemy spawn interval.

8.4 Screenshots

Figure 8.1 shows a running game session with the Greedy recommendation and Backtracking path both visible. Figure 8.2 shows the HUD panel where the BST in-order listing is rendered in real time.

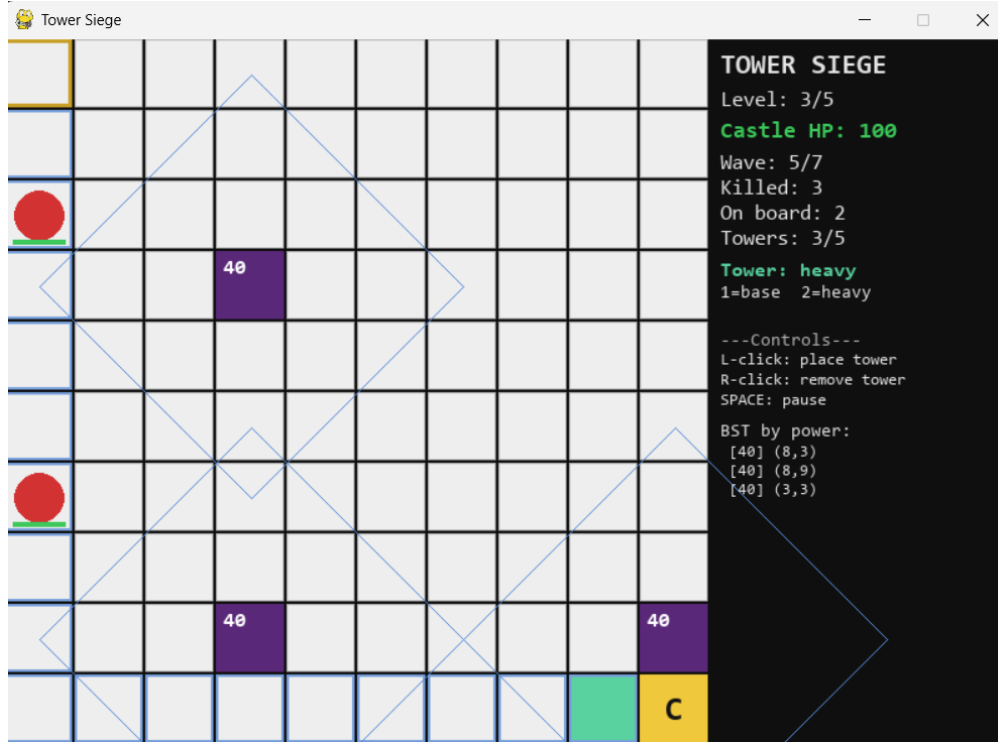


Figure 8.1: Gameplay screenshot. The teal cell is the Greedy-recommended placement; blue outlines trace the Backtracking path from the spawn point to the castle. The diamond outline around a tower shows its Manhattan attack radius.

8.5 Validation Results

Due to the interactive nature of the game the test strategy was divided into three phases:

Unit validation (informal). Each module was exercised in isolation before integration. `greedy.py` was called with hand-crafted enemy sets to verify the scoring formula; `backtracking.py` was run against small grids with known shortest paths. The C++ engine was invoked directly with manually authored JSON files to confirm list/BST operations and event emission.

Integration validation. The full pipeline (Python writes `input.json` → engine runs → Python reads `state.json`) was verified by playing all five levels to completion. Enemy HP decremented correctly across steps, the BST in-order listing matched manual inspection, and `wave_clear` events triggered level transitions.

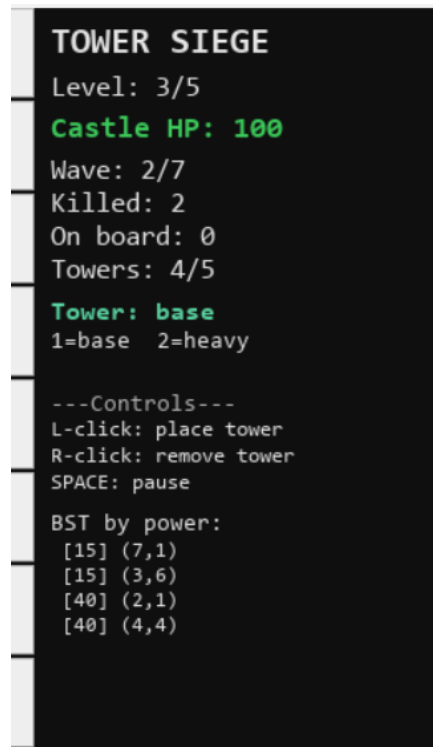


Figure 8.2: HUD panel. The “BST by power” list displays towers in ascending `attack_power` order, directly demonstrating the BST in-order traversal property at runtime regardless of placement order.

Acceptance validation. A session of each level was played under the intended use-case (player follows the greedy hint) and under an adversarial use-case (player ignores the hint and places towers sub-optimally). Both sessions terminated correctly with a win or lose outcome and no Python exceptions.

Table 8.1 summarises the validation cases.

Table 8.1: Summary of validation cases.

Category	Case	Expected / Observed
Unit	Greedy with no enemies	Returns $(0, 0)$ (default; no enemies to score) – Pass
Unit	Backtracking, open grid	Shortest path length = $(r - 1) + (c - 1)$ for goal (r, c) – Pass
Unit	Backtracking, fully blocked	Returns empty list – Pass
Unit	BST in-order traversal	Towers emitted in ascending attack_power order – Pass
Unit	Linked list advance	Enemy position increments by speed cells each step – Pass
Integ.	Engine step (no towers)	Enemy advances, no events; castle_hp unchanged – Pass
Integ.	Tower kills enemy	enemy_killed event emitted; enemy removed from list – Pass
Integ.	Enemy reaches castle	castle_damaged event; castle_hp decreases by 10 – Pass
Accept.	Level 1 win	5 kills, castle HP > 0 – Pass
Accept.	Castle falls (level 2)	lost phase shown; press R resets – Pass
Accept.	Path-block prevention	Tower that would seal spawn rejected by placing_blocks_path – Pass

9. Discussion

9.1 Greedy–Backtracking Interaction Bug

The most significant observed issue in Tower Siege arises from the interaction between the Greedy scoring function and the Backtracking pathfinder. This section analyses the root cause, the trigger conditions, and the measurable effect, and proposes two remediation strategies.

9.1.1 Root Cause: Castle-Proximity Bias in Greedy

The Greedy score for cell (r, c) includes the term

$$\max(0, 8 - d_{\text{castle}}) \cdot 1.5$$

where $d_{\text{castle}} = |r - r_{\text{castle}}| + |c - c_{\text{castle}}|$. Because this term is multiplied by a weight of 1.5 and applies to *every* cell regardless of enemy proximity, cells adjacent to the castle cell receive a bonus of up to $7 \times 1.5 = 10.5$ score units. On an empty board with few enemies, or when enemies are far from the castle, this proximity bonus dominates the threat sum and the Greedy consistently recommends cells in the bottom-right quadrant—the neighbourhood of the castle.

9.1.2 Pathfinder Exposure

As the player follows the Greedy recommendations, the bottom-right area becomes progressively filled with towers. The Backtracking DFS must then route enemies through the remaining free cells, which form an increasingly narrow corridor. Although the pruning optimisation

$$\text{len}(\text{path}) + d_{\text{Manhattan}}(\text{current}, \text{goal}) \geq \text{len}(\text{best}) \implies \text{prune}$$

eliminates many branches once a first path is found, the *first* solution may require exploring a large fraction of the search space when the best path is long and convoluted. In the worst case (a maze-like corridor of length L through an $n \times m$ grid), the DFS visits $O(4^L)$ states before finding the first path.

Furthermore, `placing_blocks_path`—the guard that prevents the player from sealing all routes—calls `find_path` on every attempted tower placement. As the grid fills, this validation itself becomes slow, amplifying the perceptible pause before each tower is accepted or rejected.

9.1.3 Observable Symptom

On a 10×10 grid in level 5 with several towers placed near the castle (as recommended by Greedy), the following behaviour occurs:

1. The player clicks a cell adjacent to an existing tower cluster.
2. `placing_blocks_path` invokes `find_path` for each of the two spawn points.
3. With a narrow remaining corridor, each DFS explores a large number of candidate paths before finding the shortest.
4. The Pygame event loop is blocked for the duration of the DFS (Python is single-threaded).
5. The game appears to **freeze** for several seconds.

If all routes to the castle happen to be blocked despite the guard (e.g. by a race condition between the spawn timing and the placement check), `find_path` returns an empty list, the engine receives an empty paths array, and enemies remain stationary—another freeze condition.

9.1.4 Proposed Remediations

Two remediation strategies are identified:

Short-term: re-weight the Greedy scoring function. Reducing the castle-proximity weight from 1.5 to 0.3–0.5 would shift recommendations toward cells along the middle of the likely enemy routes rather than near the castle. A proper influence map—precomputed by flooding the grid from each spawn point—would provide path-frequency data as a stronger signal than raw castle distance.

Long-term: replace Backtracking with A^* . Using A^* with the Manhattan heuristic would reduce worst-case path computation from $O(4^L)$ to $O(b^d)$ with $b \approx 2.5$ on average for grid graphs [2]. Because A^* is guaranteed to find the shortest path without exhaustive exploration, the `placing_blocks_path` guard would run in milliseconds rather than seconds even on highly occupied grids. This change is confined to `backtracking.py` and would not require any modification to the C++ engine or the JSON contract.

9.2 Data Structure Performance

The unbalanced BST performed adequately during testing because tower attack powers take only two distinct values (15 for base towers, 40 for heavy towers). With only two

distinct key values the tree height is bounded by 2, so all operations are effectively $O(1)$. A future variant with continuous attack-power upgrades would expose the absence of balancing and benefit from AVL or Red-Black rotations [1].

The linked list showed constant-time append and acceptably linear removal for wave sizes up to 10 enemies, consistent with the $O(n)$ theoretical bound.

10. Conclusion

Tower Siege successfully demonstrates the integration of four classical computer-science artefacts—a singly linked list, a Binary Search Tree, a Greedy heuristic, and a Backtracking DFS—within a playable, five-level tower-defense game. The strict separation between the C++ engine and the Python algorithm/UI layers, enforced through a JSON file contract, proved effective: each layer can be tested in isolation, and the contract acts as a self-documenting interface specification.

The main engineering finding is that algorithm interactions must be analysed holistically, not in isolation. Individually, the Greedy function and the Backtracking pathfinder behave correctly; together, the Greedy’s castle-proximity bias progressively exposes the Backtracking’s exponential worst-case complexity. This interaction is a concrete instance of the general principle that locally optimal decisions (Greedy placement near the castle) can globally degrade system performance (Backtracking freeze).

Future work should address three open points:

1. Re-weight or replace the Greedy scoring function using a path-frequency influence map to bias recommendations toward high-traffic corridors.
2. Replace the Backtracking DFS with A^* to eliminate the exponential worst case in path computation and placement validation.
3. Implement AVL balancing in the BST to guarantee $O(\log n)$ performance when attack-power values span a wide range.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th. Cambridge, MA: The MIT Press, 2022.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. DOI: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136)
- [3] Python Software Foundation. “Json — json encoder and decoder,” Accessed: Jun. 3, 2026. [Online]. Available: <https://docs.python.org/3/library/json.html>
- [4] N. Lohmann. “Json for modern c++,” Accessed: Jun. 3, 2026. [Online]. Available: <https://github.com/nlohmann/json>
- [5] Pygame Community. “Pygame documentation,” Accessed: Jun. 3, 2026. [Online]. Available: <https://www.pygame.org/docs/>